AI 海難救助ドローン

転覆事故の漂流者を AI で見つけ、救命浮環を届ける

Physical AI 応用講座1 最終課題 説明資料 ／ PX4 SITL + Gazebo Harmonic + ROS 2 Humble + YOLO11 ／ 動画: YouTube(提出フォームに記載) ／ コード: GitHub(提出フォームに記載)



図1 デモ動画の 1 場面。左はカメラ映像と AI の検出枠(赤: 人)で、投下した救命浮環が海面に浮いている。右上は Gazebo の 3D 画面、右下は捜索マップ(黄色の矢印: 通報地点 → 漂流を補正した捜索の中心)と状態

1. 課題の概要と目的

海で人が流されると、時間とともに通報された地点から離れていきます。早く見つけて浮力のあるものを届けることが、生存に直結します。 この課題では、**ドローンが流された位置を予測して自律的に捜索し、AI で漂流者を見つけて確かめ、救命浮環を投下する**システムを作りました。 陸上の災害を想定して作った「AI 捜索ドローン」を土台に、海特有の課題に取り組みました。海特有の課題は、対象が動くこと、捜索の起点を推定する必要があること、物を届ける必要があることです。

> **想定**:名護市シーグラスビーチの沖で釣り船が転覆し、乗員 5 名(浮いている人 2・立ち泳ぎの人 3)が海に投げ出された。 要救助者と転覆船は海流(0.15 m/s、方位 240°)で流され続けている。ドローンは**通報から 10 分後**に砂浜から出動する。 ドローンが知っているのは、通報内容(転覆地点・乗員数)と海流の予報だけ。要救助者の本当の位置は評価にしか使わない。

2. システム構成

Gazebo	→	detector	→	mission	→	PX4 SITL
海・漂流する要救助者		YOLO11m タイル推論 画素 → 海面上の位置		漂流予測・捜索 確認・投下		オフボード制御

payload	dashboard	truth_logger(評価専用)
救命浮環の投下装置 (生成・落下・漂流)	カメラ映像・地図・状態	本当の位置を記録 AI 側には渡さない

状態	動作
漂流予測	通報地点 + 海流 × (通報からの経過時間 + 到着までの時間)を捜索の基点にする
拡大方形捜索	IAMSAR の捜索パターン。基点から 1S, 1S, 2S, 2S, … と右回りに広げる。S はカメラの撮影幅の 75% (32 m)
発見・再確認	確信度 0.45 以上で候補にする。高度 10 m で再確認し、確信度 0.6 以上を 2 回検出したら確定。確定できなければ棄却する
救命浮環の投下	予測位置の真上の高度 6 m で 1.5 秒静止して投下する。終わったら捜索へ戻る。乗員 5 名を全員確定したら帰還する

3. 工夫した点

(1) 漂流予測と拡大方形捜索:どこから探すか

通報から 10 分たつと、要救助者は約 100 m 流されています。カメラが一度に見られるのは前方 50 m ほどなので、通報地点をそのまま探しても見つかりません。 捜索の基点は、海流の予報 × 経過時間だけずらしました。これで、基点と要救助者の集団の中心との距離は**約 107 m から約 15 m に縮まり**、出動から 31 秒で最初の 1 名を確定できました。 基点のまわりは、国際的な捜索救助マニュアル(IMO・ICAO の IAMSAR マニュアル)にある拡大方形捜索で、近い所から順に外側へ探します。

(2) 動き続ける対象の追跡と、救命浮環の投下

見つけた人も、確かめている間や投下の準備中に流されていきます。そこで候補は「推定した位置と時刻」の組として持ちます。 照合・接近・投下では「経過時間 × 海流」を足した**予測位置**を使い、確定位置は近距離での観測を同じ時刻に揃えてから平均しました。 投下装置(`payload` ノード)は、機体の下に救命浮環を出現させ、自由落下($z = h - g\,t^2/2$)させて海面に浮かべます。その後は、浮環も海流で流します。 Gazebo には浮力の計算が無いため、浮環は重力を切ったモデルにし、動きをノードから与えています。

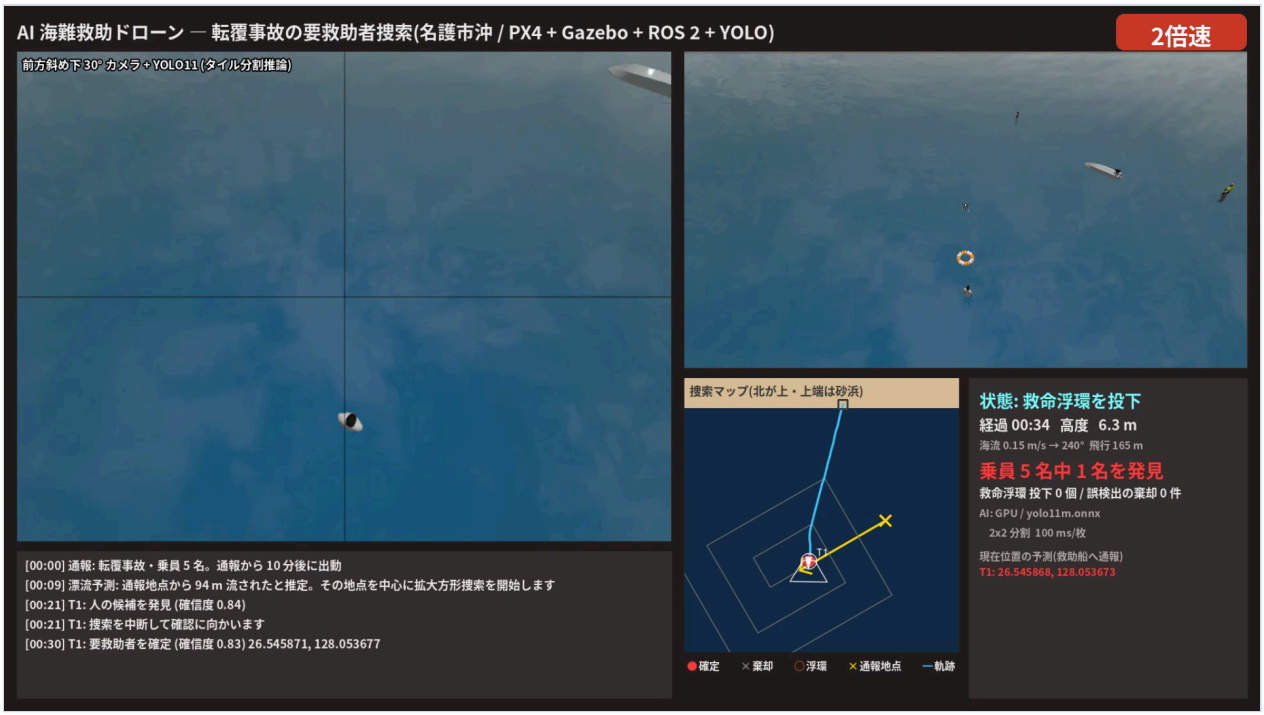


図2 救命浮環の投下。高度 6 m で静止して投下し、右上の 3D 画面で浮環(オレンジ)が要救助者の横に浮いている

(3) 陸上版から引き継いだ工夫

- **タイル分割推論**:上空からは人が十数画素にしか写らないので、画像を 3×3 に分けて推論する。全体を 1 回で推論する方式では、1 人も検出できなかった。
- **カメラを 30° 前へ傾けた機体**:真下向きでは、立っている人を 6 名中 3 名見逃した。斜めにすると全員を検出できた。海でも、胸まで水中の人を確信度 0.5 以上で検出できる(実験で確認)。
- **2 段階確認**:ブイや浮遊物の誤検出は、降下して再確認する段階で棄却する。
- **姿勢を補正した位置推定**:画像を受け取った時点の姿勢で、視線と海面の交点を求める。

(4) 公平な評価

評価用の `truth_logger.py` が、Gazebo から要救助者と浮環の本当の位置を 0.5 秒ごとに記録します。ミッションの報告書とは時刻で照合しています。 要救助者は動いているので、評価用のスクリプトは確定・投下の時刻の位置を補間して比べます。この記録は AI と制御には一切使っていません。

4. 結果

表1 海難救助ミッションの評価(乗員 5 名、同じ条件で 3 回飛行。3 回目がデモ動画)

	1回目	2回目	3回目
発見した要救助者	**5 / 5**	**5 / 5**	**5 / 5**
誤った確定 / 誤検出として棄却	0 / 0	0 / 1	0 / 1
確定位置の誤差 平均 / 最大	0.45 / 0.77 m	0.49 / 0.92 m	0.65 / 0.99 m
救命浮環と要救助者の距離 平均 / 最大	**0.51 / 0.87 m**	**0.45 / 1.07 m**	**0.65 / 1.19 m**
最初の 1 名 / 全員を確定するまで	31 / 161 s	31 / 174 s	31 / 189 s
飛行時間 / 飛行距離	216 s / 1021 m	229 s / 1024 m	240 s / 1094 m

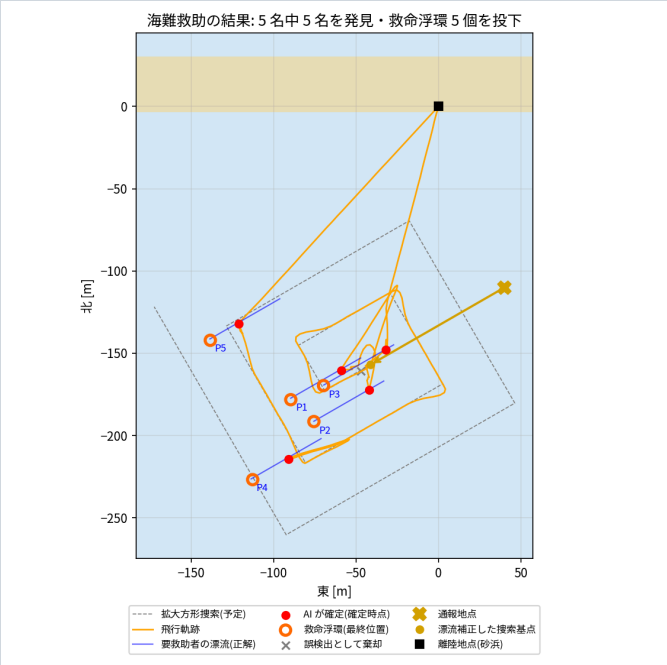


図3 3 回目の結果。青線は要救助者の漂流(本当の位置)、赤点は AI が確定した位置、オレンジの輪は救命浮環。黄色の矢印は、通報地点から漂流を補正した捜索の中心へ

3 回とも、乗員 5 名全員を誤りなく発見しました。救命浮環は **15 個すべてが要救助者から 1.2 m 以内**に落ちました。手を伸ばして届く目安の 2 m 以内に収まっています。 ブイなどの誤検出の候補は、降下して再確認する段階で棄却できました。

漂流補正がなければ、最初の数分は通報地点のまわりを空振りで探すことになります。補正した基点から探し始めたので、最初の 1 名を 31 秒で確定でき、全員の確定まで 3 分ほどでした。

遅く見つかった人ほど位置の誤差がやや大きい(最大 0.99 m)傾向がありました。 海流は設定どおりの速さ(0.150 m/s)で流れていることを確認したので、漂流予測のずれではありません。原因の切り分けは今後の課題です。

5. 考察と今後の課題

- **海流の不確かさ**:今回は、予報した海流と実際の海流が一致する条件で試しました。実際の海では予報に誤差があるので、 漂流の推定範囲を確率分布で表し、確率の高い所から探す捜索計画(米国沿岸警備隊の SAROPS などの考え方)に広げたいと考えています。
- **見え方の変化**:波・白波・太陽の照り返し・夜間では、人の見え方が大きく変わります。海上から撮影したデータで追加学習することや、 赤外線カメラを組み合わせることが必要です。
- **投下の物理**:風による浮環の流され方や、着水後の波による動きは表現していません。風の影響を見込んで、投下位置を風上側にずらす制御が考えられます。
- **運用**:確定位置(緯度・経度と漂流予測)は JSON で出力しています。この出力を救助船への送信や、複数機での分担捜索につなげられます。

6. 使用した技術

PX4-Autopilot v1.18(SITL、自作の機体設定 4099)、Gazebo Harmonic 8(VelocityControl、ShaderParam による海面)、ROS 2 Humble、 Micro XRCE-DDS、ros_gz_image、gz-transport(Python)、Ultralytics YOLO11m / YOLO11s(COCO 学習済み、ONNX に変換。追加の学習はしていない)、 ONNX Runtime GPU 1.20、OpenCV、Pillow、matplotlib、Gazebo Fuel のモデル(人・ボート・桟橋・漁船・ブイ・海面)。 検証環境は Ubuntu 22.04 と NVIDIA RTX 3060 Laptop GPU です。